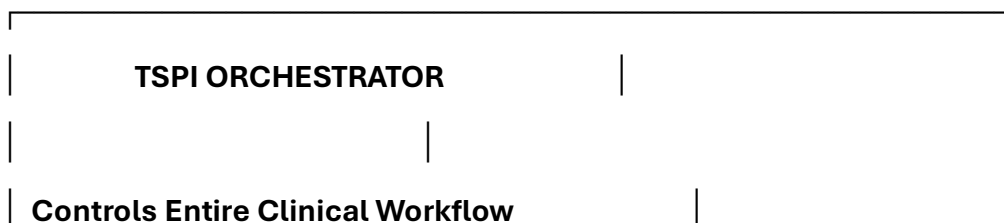
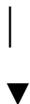
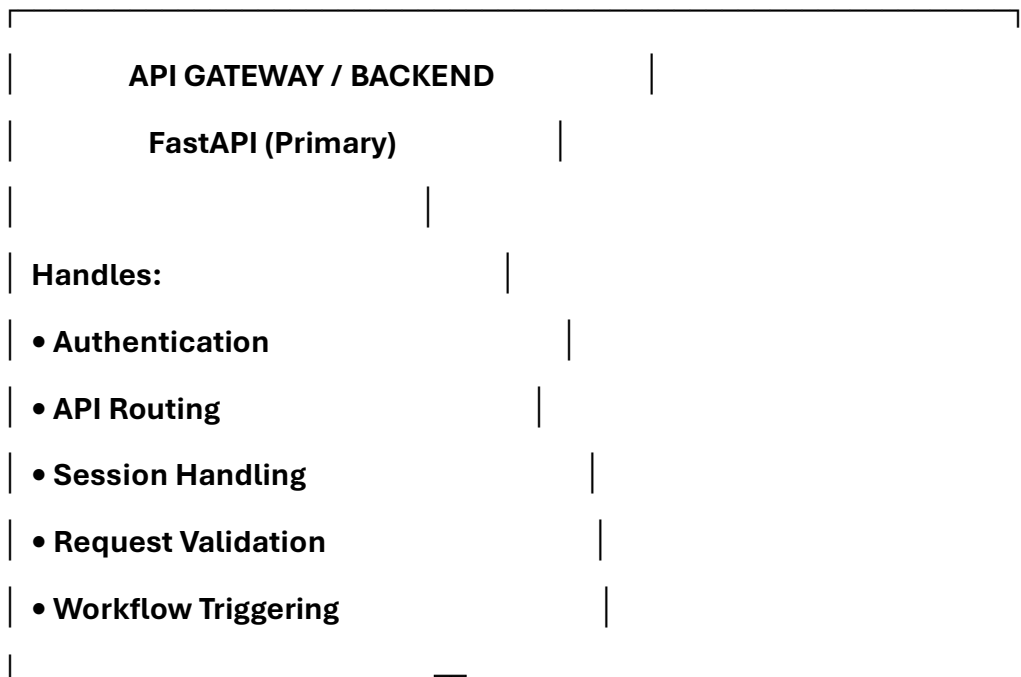
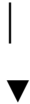
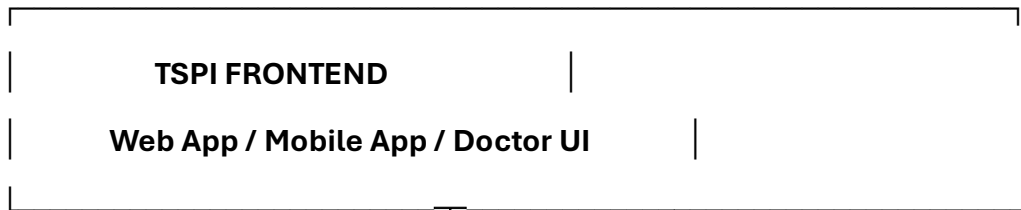
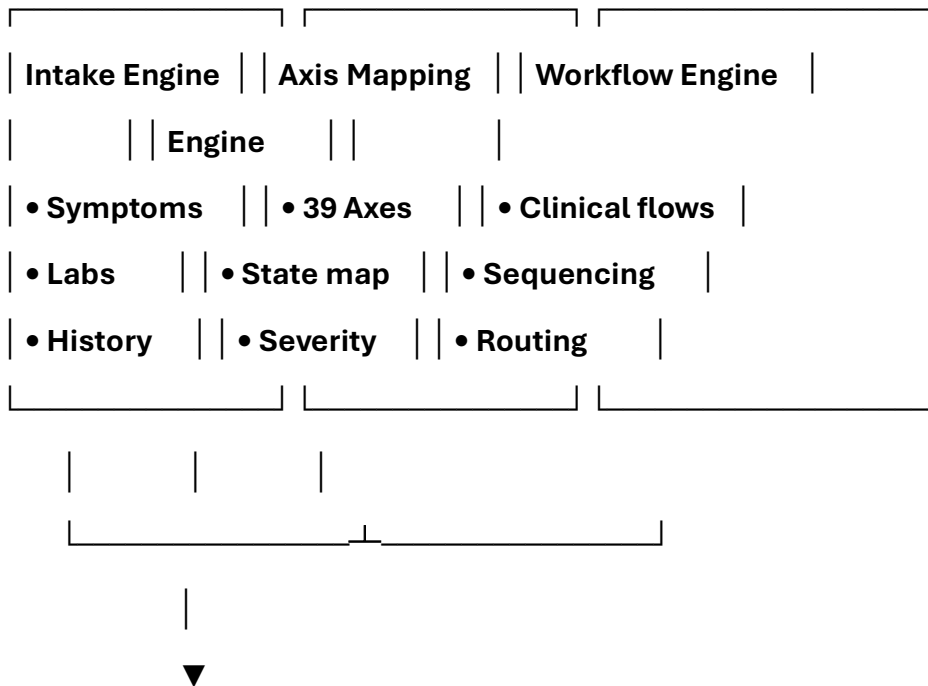
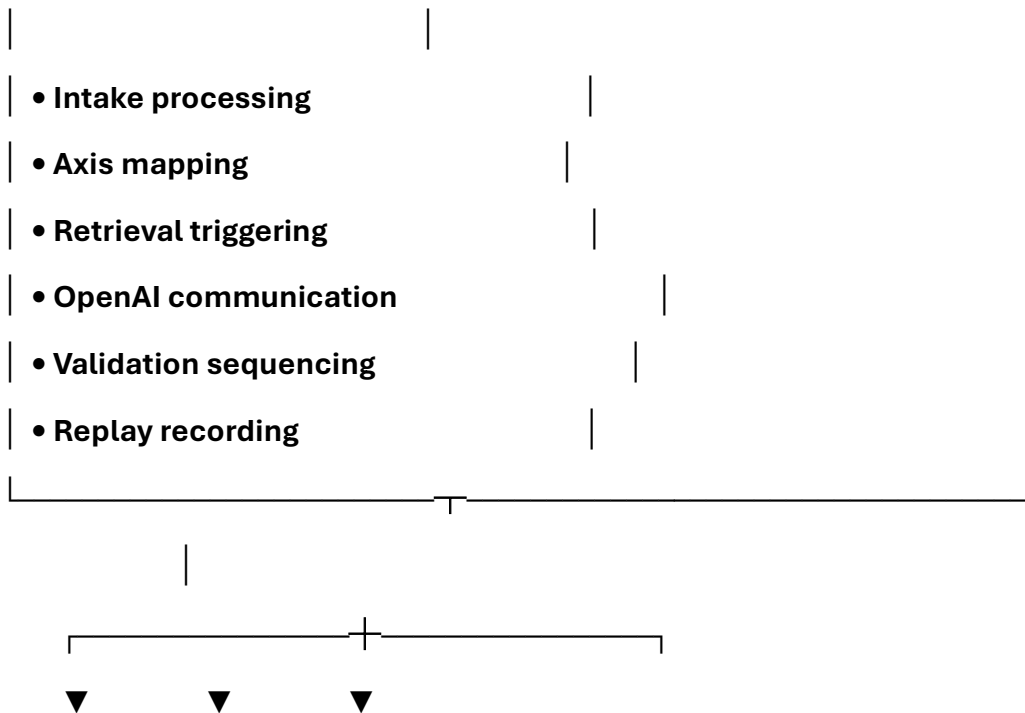
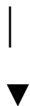
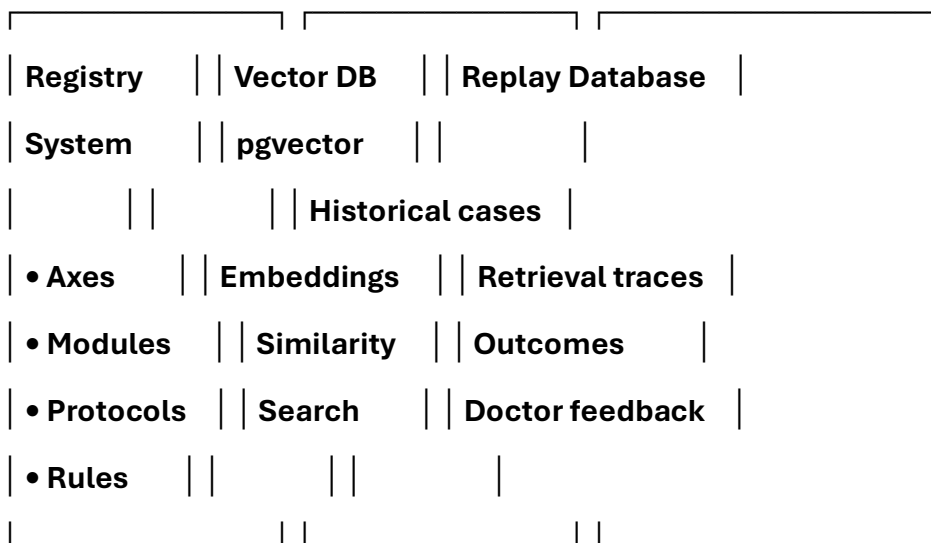
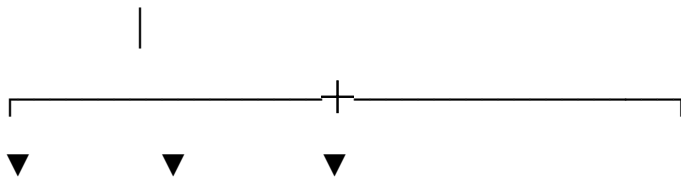
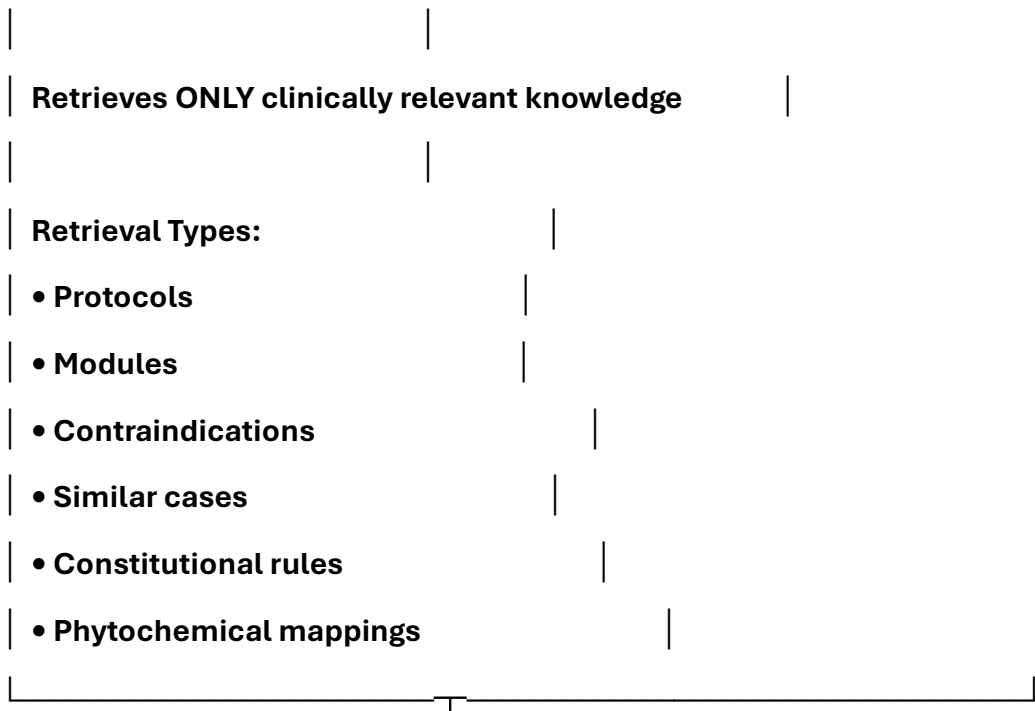


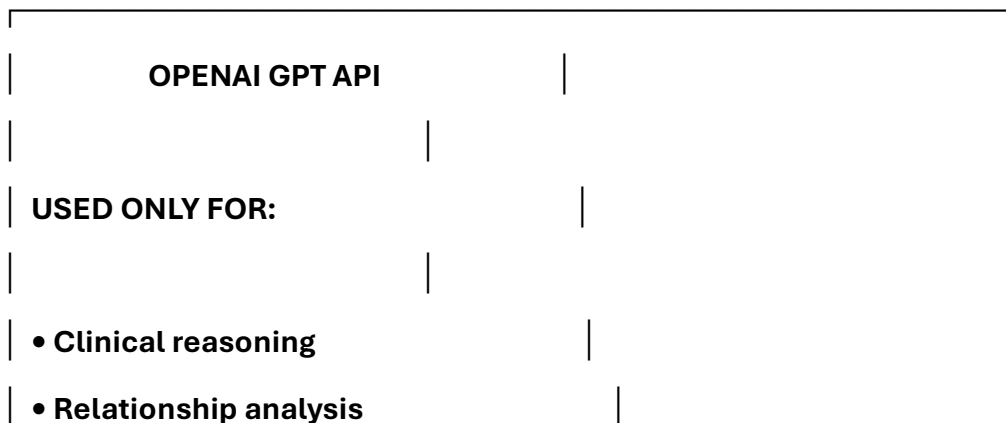
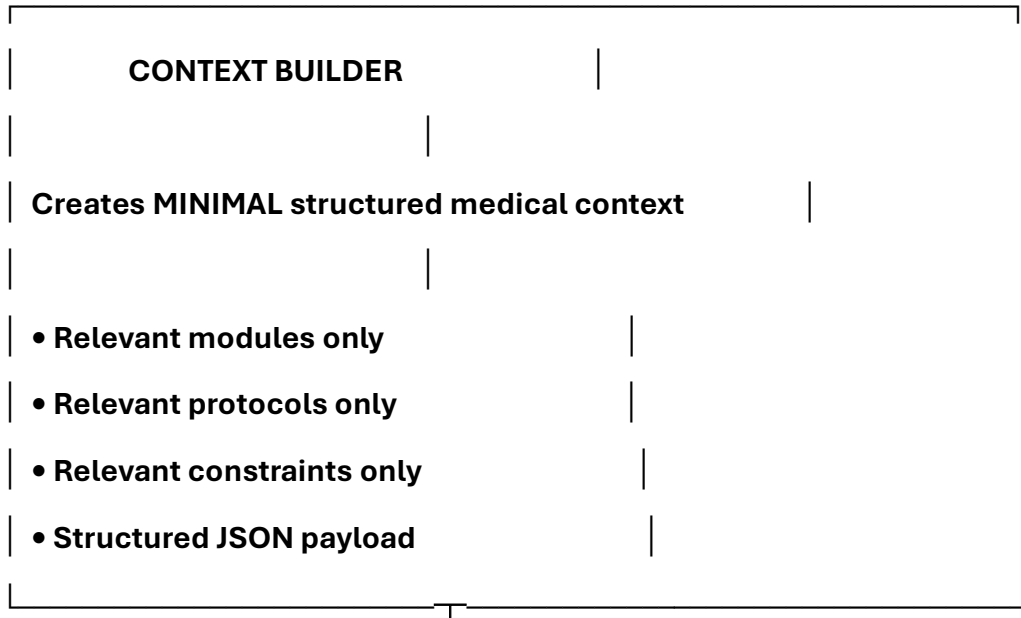
TSPI Current Recommended Hybrid Architecture (Practical MVP)





CONTROLLED MEDICAL RAG ENGINE





- Structured synthesis
- Explanation generation
- Recommendation formatting

OPENAI DOES NOT CONTROL:

- Governance
- Retrieval
- Protocol rules
- Validation
- Replay
- Constitutional logic

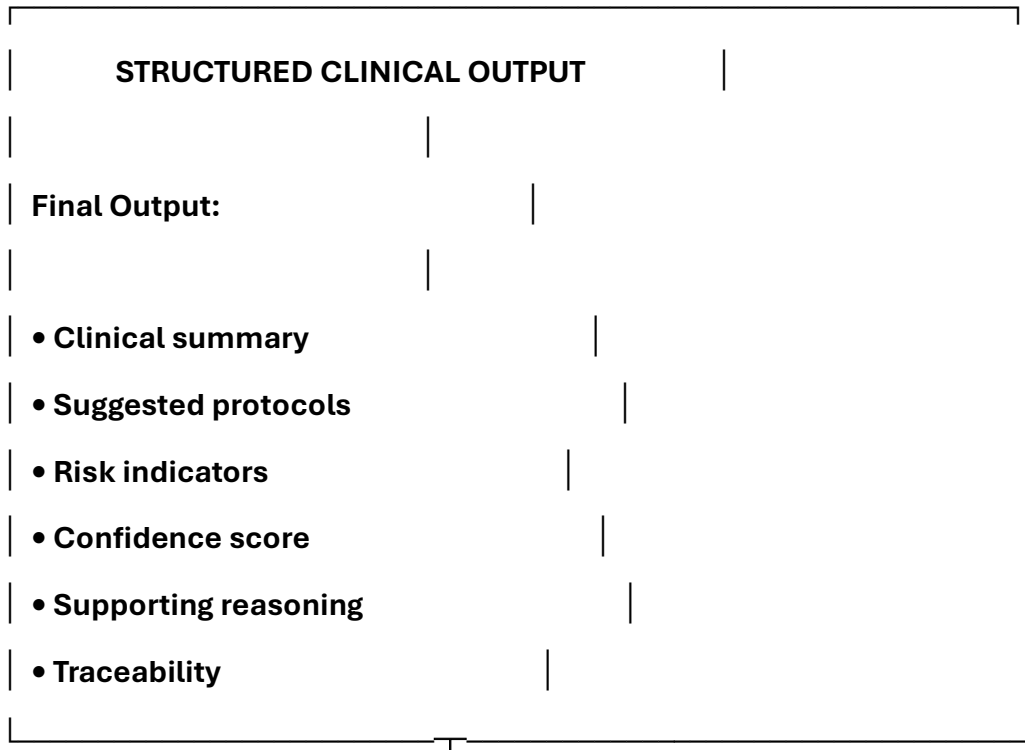
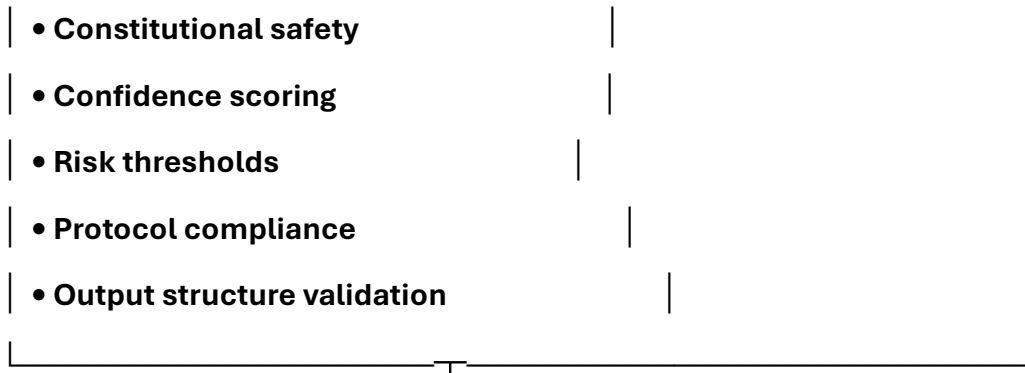


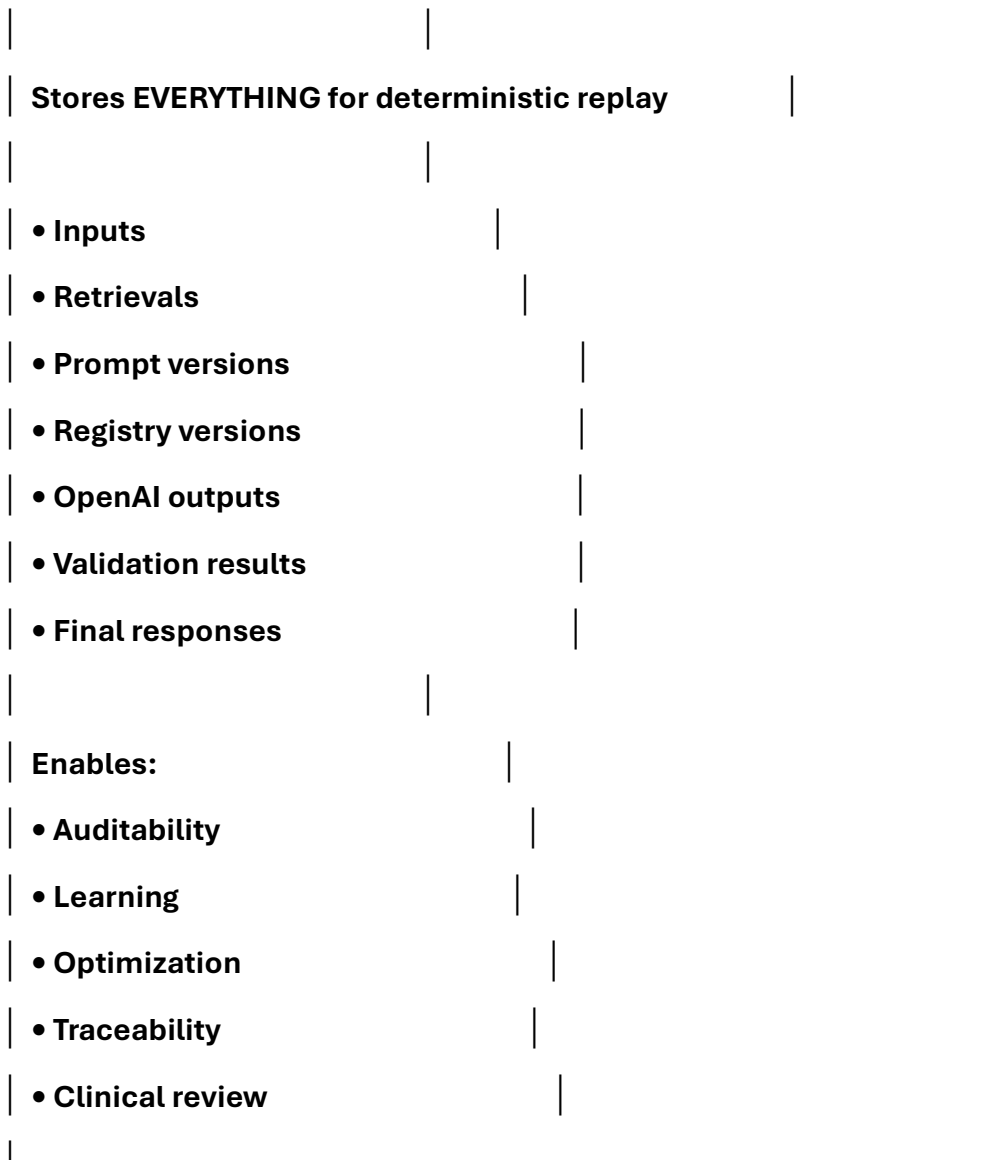
RESPONSE RETURNS TO TSPI

TSPI VALIDATION LAYER

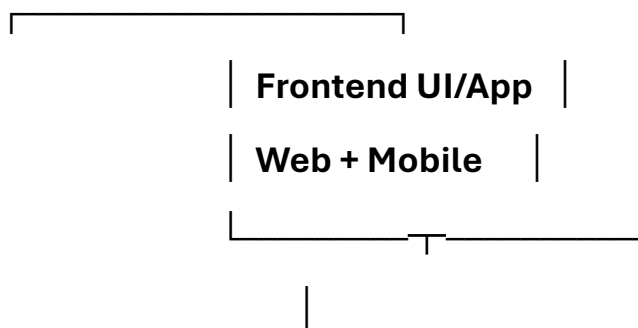
TSPI verifies ALL outputs

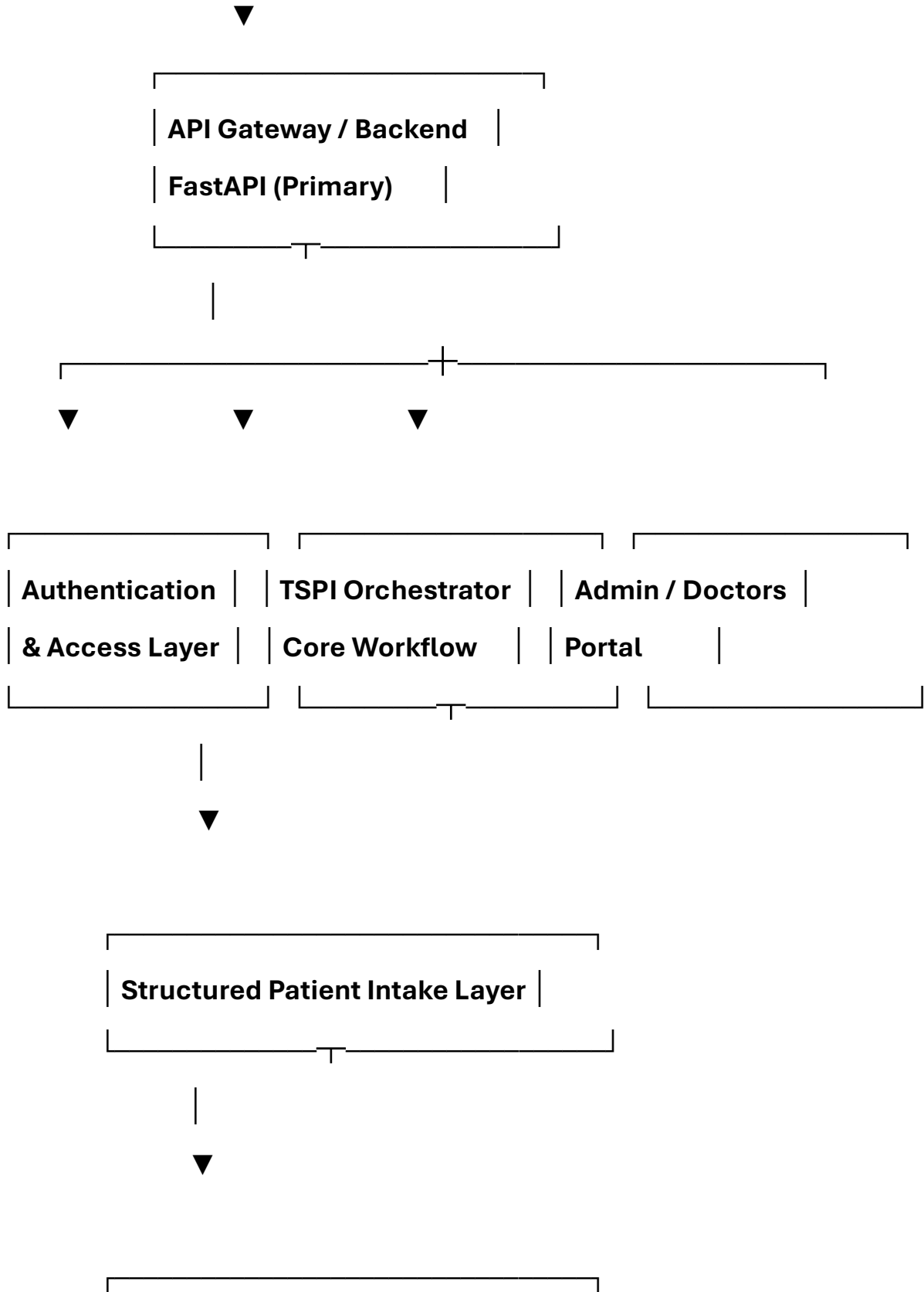
- Contraindication checks





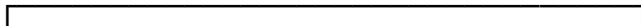
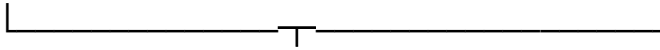
High-Level TSPI Architecture





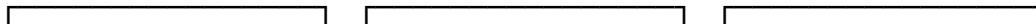
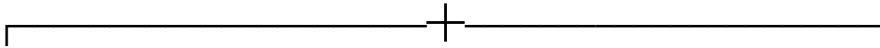
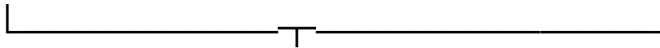
| **Axis Mapping & State Engine** |

| **(TSPI Proprietary Logic)** |



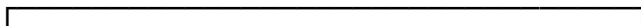
| **Retrieval Engine (Controlled** |

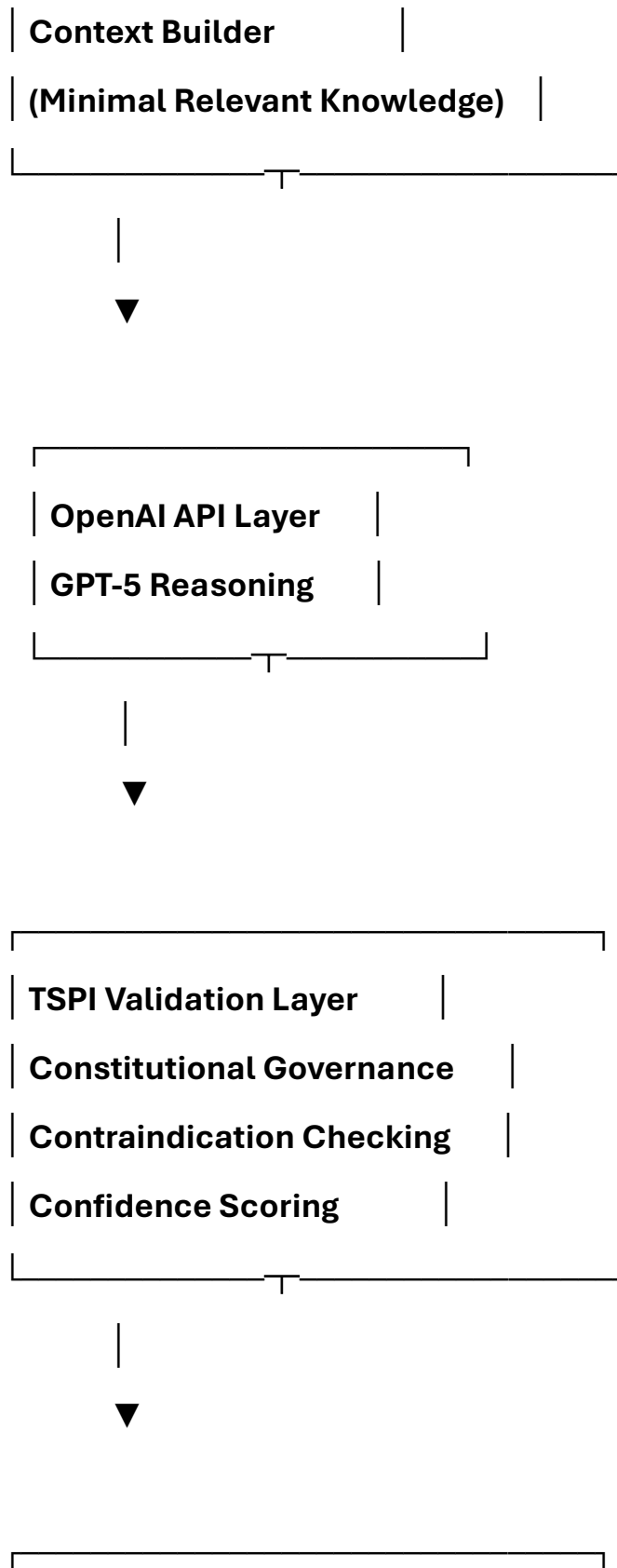
| **Medical RAG)** |

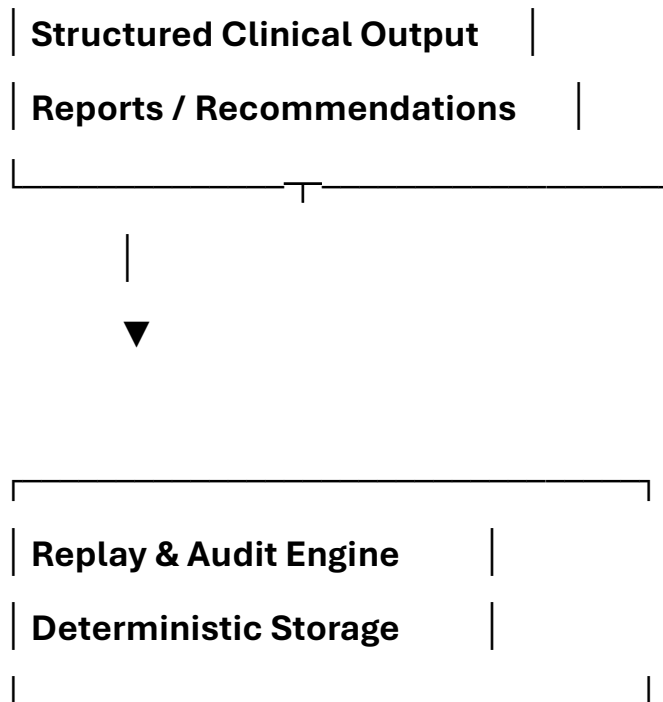


| **Registry** | | **Vector DB** | | **Replay History** |

| **System** | | **pgvector** | | **Similar Cases** |







This architecture keeps:

- ✓ TSPI proprietary intelligence
- ✓ replay determinism
- ✓ constitutional governance
- ✓ controlled medical RAG
- ✓ OpenAI reasoning
- ✓ future scalability

Where OpenAI Is Used

This is VERY important.

OpenAI is ONLY used for:

Task

OpenAI?

Clinical reasoning

YES

Pattern synthesis

YES

Medical summarization

YES

Explanation generation

YES

Structured output generation

YES

What OpenAI DOES NOT Control

Task

TSPI

Governance

YES

Retrieval

YES

Protocol rules

YES

Axis mapping

YES

Safety constraints

YES

Validation

YES

Replay

YES

Registries

YES

Scoring

YES

This is the core hybrid model.

Complete End-to-End TSPI Flow

STEP 1 — Patient Input

Patient enters:

Symptoms

Lab values

Medical history

Medications

Lifestyle

Doctor notes

Example:

JSON

{

 “symptoms”: [“fatigue”, “poor sleep”],

 “labs”: {

“ALT”: 78

}

}

STEP 2 — Structured Intake Layer

TSPI converts raw input into:

Normalized medical structure

Symptom ontology

Metadata tags

Example:

JSON

{

“normalized_symptoms”: [

“sleep_dysfunction”,

“fatigue”

]

}

STEP 3 — Axis Mapping Engine (TSPI Brain)

TSPI proprietary logic maps:

Symptoms → axes

Conditions → modules

Risks → constraints

Example:

JSON

```
{  
  "axes": [  
    "inflammation_axis",  
    "recovery_axis"  
  ]  
}
```

This is YOUR intelligence layer.

STEP 4 — Controlled Medical RAG Retrieval

TSPI retrieves ONLY relevant knowledge.

From:

Protocols

Modules

Contraindications

Historical cases

Scoring rules

Example retrieval:

JSON

```
{  
  "retrieved_modules": [  
    "liver_recovery_v2",  
    "sleep_support_v1"  
  ]  
}
```

```
}
```

STEP 5 — Context Builder

TSPI creates:

Minimal structured context

NOT giant prompts.

Example:

JSON

```
{  
  "patient_state": {},  
  "axes": [],  
  "retrieved_protocols": [],  
  "contraindications": []  
}
```

This minimizes:

Hallucinations

Token cost

Privacy exposure

STEP 6 — OpenAI Reasoning Layer

NOW OpenAI is called.

This is where TSPI connects to:

OpenAI GPT models

OpenAI receives:

structured patient state

retrieved knowledge

governance instructions

output schema

OpenAI Task Example

Plain text

Analyze relationships between:

- inflammation axis
- fatigue
- elevated liver markers

Use only retrieved protocols.

Return structured JSON.

OpenAI performs:

reasoning

synthesis

explanation

analysis

STEP 7 — TSPI Validation Layer

After OpenAI responds:

TSPI validates:

contraindications

unsafe recommendations

constitutional conflicts

confidence thresholds

workflow compliance

This is CRITICAL.

STEP 8 — Structured Output

TSPI returns:

Clinical summary

Protocol candidates

Risks

Confidence score

Reasoning trace

Example:

JSON

```
{  
  "summary": "",  
  "recommendations": [],  
  "confidence": 0.81  
}
```

STEP 9 — Replay Engine

Everything is stored:

JSON

```
{  
  "input": {},  
  "retrievals": [],  
}
```

```
“prompt_version”: “”,  
“registry_version”: “”,  
“model”: “gpt-5”,  
“output”: {},  
“validation”: {}  
}
```

This enables:

Replay

Auditing

Learning

Debugging

Future optimization

TSPI Core Proprietary Systems

1. Registry System

This becomes:

Your medical brain

Structure:

Plain text

Registry/

└─ axes/

└─ modules/

└─ protocols/

└─ compounds/

└— contraindications/

└— workflows/

└— constitutional_rules/

2. Replay System

This becomes:

Your learning memory

Stores:

Outcomes

Retrievals

Failures

Doctor feedback

Protocol effectiveness

3. Validation Layer

This becomes:

Your safety layer

Checks:

Unsafe combinations

Confidence thresholds

Protocol eligibility

4. RAG Layer

This becomes:

Your intelligence delivery system

Feeds OpenAI:

The right medical knowledge

At the right time

Recommended Technology Stack

Layer	Recommendation
Backend.	FastAPI
Database	PostgreSQL
Vector D	BPgvector
AI Reasoning.	OpenAI
Queue.	Redis
Frontend	React/Next.js
Containers.	Docker

Docker Container Structure

Plain text

Frontend

Backend-api

Worker

Postgres

Redis

Optional later:

Plain text

Qdrant

Monitoring

Analytics

Future Evolution Path

Phase 1 (NOW)

Plain text

OpenAI

+

TSPI Registry

+

Controlled RAG

+

Replay

Phase 2

Add:

Protocol ranking

Case similarity

Retrieval optimization

Phase 3

Add:

Specialized local models

Classifiers

Scoring models

Phase 4

Optional:

Fine-tuned TSPI models

Hybrid inference

Private reasoning systems

ONLY later.

MOST Important Architectural Principle

TSPI should own:

Governance

Retrieval

Validation

Workflows

Intelligence structure

While OpenAI provides:

Reasoning capability

That is the correct hybrid architecture.